

ЛАБОРАТОРНА РОБОТА 1 (частина 2) НАЛАШТУВАННЯ CI/CD ПАЙПЛАЙНУ ТА ДОКУМЕНТУВАННЯ КОДУ НА GITHUB

Мета роботи: ознайомитися з концепціями безперервної інтеграції (Continuous Integration) та безперервного розгортання (Continuous Deployment); здобути практичні навички налаштування автоматизованих пайплайнів для тестування та перевірки якості коду за допомогою GitHub Actions; опанувати основи безпеки (GitHub Secrets) та оформлення проєктної документації за допомогою Markdown.

Обладнання:

- ПК з встановленою операційною системою;
- доступ до мережі інтернет;
- встановлений клієнт Git та локальне середовище розробки (редактор коду);
- обліковий запис на платформі GitHub.

1.1 Теоретичні відомості

Безперервна інтеграція (Continuous Integration, CI) — це практика розробки програмного забезпечення, яка полягає у частому злитті робочих копій коду розробників у спільну головну гілку. Основна мета CI — швидке виявлення та виправлення помилок завдяки автоматизованому збиранню проєкту та запуску тестів після кожного коміту.

Безперервне розгортання (Continuous Deployment, CD) автоматизує розгортання коду у тестовому середовищі або безпосередньо у production середовищі після успішного проходження CI.

GitHub Actions — це платформа автоматизації робочих процесів (workflows) безпосередньо у репозиторії GitHub. Вона дозволяє створювати пайплайни, реагуючи на події в репозиторії (наприклад, push, pull_request). Конфігурація пайплайну описується мовою YAML (файли .yaml) і обов'язково повинна зберігатися у

прихованій директорії `.github/workflows/` в корені проєкту. Структура конфігураційного файлу YAML:

- `name:` — назва робочого процесу.
- `on:` — тригери (Triggers). Визначає події, що запускають пайплайн. Щоб пайплайн запускався для конкретних гілок, використовують фільтр `branches:` ["main"]. Якщо фільтр прибрати, тести будуть запускатися при коміті у будь-яку гілку.
- `jobs:` — перелік робіт (наприклад, `build`, `test`), які можуть виконуватися паралельно або послідовно.
 - `runs-on:` — вказує операційну систему віртуальної машини (Runner), наприклад `ubuntu-latest`.
 - `steps:` — покрокова інструкція для віртуальної машини.
 - `uses:` — застосування готових скриптів (Actions), наприклад `actions/checkout@v4` для завантаження коду на Runner або `actions/setup-python@v5` для розгортання середовища.
 - `run:` | — виконання консольних команд (bash скриптів). Символ | дозволяє вводити багаторядкові команди.

Статичний аналіз коду (Linting). Забезпечення якості включає перевірку коду на відповідність стандартам оформлення (наприклад, PEP8 для Python). Для цього використовують лінтери (наприклад, `flake8`), які автоматично перевіряють відступи, порожні рядки та синтаксис.

Безпека (GitHub Secrets). Паролі та ключі API заборонено зберігати у відкритому коді. Натомість їх додають у налаштування репозиторію як Secrets. Під час виконання пайплайну GitHub автоматично розпізнає їх і маскує зірочками `***` у логах. Звернутися до Secrets в YAML можна через синтаксис `${{ secrets.НАЗВА_СЕКРЕТУ }}`.

Оформлення документації (Markdown). Файл `README.md` — це "обличчя" проєкту. Він пишеться мовою полегшеної розмітки Markdown:

- `#` Заголовок — створення заголовка;
- `**текст**` — жирний шрифт;
- `- пункт` — маркований список;

- `![Текст](URL)` — вставка зображень або динамічних бейджів статусу пайплайну.

Управління проєктами та вимогами з використанням GitHub Issues та Projects. У сучасній розробці ПЗ код ніколи не пишеться "просто так" — кожен рядок коду має вирішувати певну проблему (баг) або реалізовувати нову функцію (фічу).

У попередніх лабораторних роботах Ви познайомились з зовнішніми системи Jira та Trello. Проте GitHub має власні потужні аналоги, які тісно інтегровані з репозиторієм:

- GitHub Issues — це аналог задач (Tasks/Bugs/Stories) у Jira. Тут розробники описують суть проблеми, прикріплюють скріншоти, ведуть обговорення та призначають відповідальних;

- GitHub Projects — це інструмент візуалізації робочого процесу, аналог Kanban-дощок у Trello або Jira. Він дозволяє організувати Issues у колонки (наприклад: To Do, In Progress, Done).

GitHub вміє автоматично закривати задачі, коли код зливається у головну гілку. Для цього в описі Pull Request або коміту достатньо написати спеціальне ключове слово та номер задачі, наприклад: Fixes #3 або Resolves #5.

1.2 Порядок виконання роботи

Етап 1. Підготовка репозиторію та коду

1 Використайте будь-який репозиторій Git. Переконайтеся, що ви знаходитесь на головній гілці (main), і стягніть останні зміни командою `git pull`.

2 Створіть нову локальну гілку: `git checkout -b feature-cicd-pipeline`.

3 Додавання коду. Ви можете обрати один із двох варіантів виконання роботи:

- варіант А (використання власного проєкту): якщо у вашому репозиторії вже є робочий програмний код (наприклад, мовами Java, C#, JavaScript, C++) та написані до нього модульні тести, ви можете налаштувати CI/CD безпосередньо для нього. Переконайтеся, що код запускається локально;

- варіант Б (використання базового шаблону на Python): якщо готового коду немає, створіть файл `calculator.py` з

навмисними дрібними порушеннями форматування:

```
def add(x, y):  
    return x + y  
def subtract(x, y):  
    return x - y
```

Створіть до нього файл тестів test_calculator.py:

```
import unittest  
from calculator import add, subtract
```

```
class TestCalculator(unittest.TestCase):  
    def test_add(self):  
        self.assertEqual(add(2, 3), 5)  
    def test_subtract(self):  
        self.assertEqual(subtract(5, 2), 3)
```

```
if __name__ == '__main__':  
    unittest.main()
```

Етап 2. Налаштування безпеки (Secrets) та захисту гілок

5. Перейдіть на сторінку репозиторію на GitHub. Відкрийте Settings -> Secrets and variables -> Actions. Натисніть New repository secret. Створіть секрет із назвою MY_SUPER_SECRET_KEY та будь-яким текстовим значенням.

6. Перейдіть у Settings -> Branches -> Add branch protection rule. Вкажіть гілку main. Поставте галочку Require status checks to pass before merging (це заблокує злиття коду, якщо тести провалені). Збережіть зміни.

Етап 3. Створення CI-пайплайну

7. Локально у корені проєкту створіть директорію .github/workflows/ та файл python-app.yml у ній.

8. Додайте наступну конфігурацію (уважно слідкуйте за відступами):

```
name: Python CI Application
```

```
on:
```

```
  push:
```

```

    branches: [ "main" ]
pull_request:
  branches: [ "main" ]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Python 3.10
        uses: actions/setup-python@v5
        with:
          python-version: "3.10"

      - name: Lint code with flake8
        run: |
          pip install flake8
          flake8 .

      - name: Demonstrate GitHub Secrets
        env:
          SUPER_SECRET: ${ secrets.MY_SUPER_SECRET_KEY }
        run: |
          echo "Спроба вивести секрет у консоль:"
          echo "Мій секрет: $SUPER_SECRET"

      - name: Run unit tests
        run: |
          python -m unittest test_calculator.py

```

Примітка для варіанту А: якщо ви використовуєте власний проєкт іншою мовою програмування, вам необхідно адаптувати файл `python-app.yml`. Наприклад, для Java змініть крок `setup-python` на `actions/setup-java`, а замість `python -m unittest...` пропишіть команду запуску ваших тестів (наприклад, `mvn test` або `npm test`).

9. Зробіть коміт усіх файлів (`git add .` та `git commit -m "Add CI pipeline and code"`).

10. Відправте гілку на сервер: `git push origin feature-cicd-pipeline` (для авторизації використовуйте *Personal Access Token* замість пароля). Створіть Pull Request у гілку main.

Етап 4. Аналіз роботи CI та виправлення помилок

11. На сторінці Pull Request ви побачите, що пайплайн завершився з помилкою (червоний хрестик), а злиття заблоковано правилом захисту гілок.

12. Перейдіть у деталі роботи (Details), відкрийте крок Lint code with flake8. Ознайомтеся з помилками форматування (нестача порожніх рядків).

13. Локально виправте код у файлах (додайте по 2 порожні рядки перед функціями та класами за стандартом PEP8). Зробіть коміт і знову відправте зміни на сервер (`git push`).

14. Дочекайтеся зеленої галочки на GitHub. Відкрийте логи успішного пайплайну та перевірте крок Demonstrate GitHub Secrets. Переконайтеся, що значення секрету приховано зірочками ***.

Етап 5. Документування (README.md)

15. На сторінці успішного пайплайну (вкладка Actions) натисніть ... та оберіть Create status badge. Скопіюйте Markdown-код.

16. Створіть або оновіть файл README.md у корені локального репозиторію. Додайте туди:

- заголовок проєкту;
- скопійований код бейджа;
- короткий опис та функціонал (маркований список);
- інструкцію із запуску тестів (блок коду ````bash`).

17. Зробіть коміт, відправте на сервер та здійсніть злиття (Merge Pull Request) у гілку main.

Етап 6. Управління проєктами та вимогами з використанням GitHub Issues та Projects

18 Процес створення Kanban-дошки (GitHub Project) описано нижче.

1. Перейдіть на головну сторінку вашого репозиторію на GitHub.

2. Відкрийте вкладку Projects (знаходиться у верхньому меню поруч із Actions та Security).

3. Натисніть кнопку Link a project, а потім New project.

4. У вікні шаблонів оберіть Board (Kanban-дошка) і натисніть Create.

5. Переіменуйте проєкт (наприклад, на назву вашої програмної системи). Ви побачите стандартні колонки: Todo, In Progress, Done.

19 Сформууйте задачі (Issues) відповідно до етапів.

6. Поверніться до репозиторію та відкрийте вкладку Issues. Натисніть New issue.

7. Сформууйте Product Backlog, створіть 3 різні задачі. Для кожної з них:

- вкажіть зрозумілу назву;
- в описі деталізуйте задачу (можна використати формат User Story: "Як користувач, я хочу...");
- у правому меню Assignees призначте цю задачу на себе;
- у меню Labels оберіть відповідну мітку (наприклад, enhancement для нового функціоналу або bug для помилки);
- у меню Projects прив'яжіть задачу до щойно створеної Kanban-дошки.

8. Перейдіть на вашу Kanban-дошку (Projects) і переконайтеся, що всі створені задачі з'явилися у колонці Todo. Перетягніть одну із задач у колонку In Progress.

20 Зв'язування коду із задачею (автоматичне закриття).

9. Зверніть увагу на порядковий номер задачі, яка зараз знаходиться у вас в In Progress (наприклад, це задача #2).

10. Локально на комп'ютері створіть нову гілку для виконання цієї задачі: `git checkout -b feature/issue-2`

11. Внесіть будь-яку логічну зміну до коду (наприклад, додайте коментар, нову функцію або оновіть README.md).

12. Зробіть коміт, обов'язково вказавши номер задачі: `git commit -am "Реалізовано функцію X (Fixes #2)"`

13. Відправте гілку на сервер: `git push origin feature/issue-2`.

14. На GitHub створіть Pull Request у гілку main. В описі (description) PR обов'язково напишіть фразу: Resolves #2 (де 2 —

номер вашої задачі).

15. Здійсніть злиття (Merge Pull Request).

16. Поверніться до вкладки Issues. Ви побачите, що задача #2 була закрита системою автоматично. Перейдіть на Kanban-дошку — ця задача також автоматично перемістилася у колонку Done.

1.3 Зміст звіту

Звіт має містити:

- титульний аркуш;
- мету роботи і завдання;
- короткий опис налаштованого CI-пайплайну та роз'яснення секцій у файлі `python-app.yml`;
- копії екранів (скріншоти) з результатами:
 1. провалений статус через `flake8` з логом помилок;
 2. лог кроку із замаскованим секретом (***);
 3. заблокована кнопка Merge Pull Request (через правила захисту);
 4. головна сторінка репозиторію з оформленим `README.md` та зеленим бейджем статусу;
 5. створена Kanban-дошка у вкладці Projects, де видно кілька задач у різних колонках (Todo, In Progress, Done);
 6. відкрита та автоматично закрита задача (Issue) на GitHub, де в історії активності видно позначку, що задачу було закрито конкретним комітом або Pull Request-ом.
- висновки щодо ефективності використання CI/CD та лінтерів.

Запитання для самоконтролю

1. Що таке Continuous Integration та які її основні переваги?
2. Що таке пайплайн і з яких типових кроків він складається?
3. Поясніть призначення ключових слів `on`, `jobs` та `steps` у конфігурації YAML.
4. Яку роль відіграють відступи (пробіли) у синтаксисі файлів YAML?
5. Для чого потрібен крок `actions/checkout@v4`?

6. Що таке лінтер (наприклад, flake8) і чому його варто запускати перед тестами?

7. Як GitHub Actions захищає конфіденційні дані (паролі, токени) під час виконання скриптів?

8. Яким чином налаштування "Branch Protection Rules" запобігає появі неробочого коду в головній гілці?

9. Для чого використовується файл README.md та яка мова розмітки в ньому застосовується?

Рекомендована література

1. Штаєр Л.О., Белей О.І. Технології розробки програмного забезпечення: лабораторний практикум. – Івано-Франківськ: ІФНТУНГ, 2025. – 178 с. – Режим доступу: https://search.library.nung.edu.ua/DocDescription?doc_id=479186

2. Understanding GitHub Actions // GitHub Docs. – Режим доступу: <https://docs.github.com/en/actions/learn-github-actions/understanding-github-actions>

3. Fowler M. Continuous Integration [Електронний ресурс] / Martin Fowler. – Режим доступу: <https://martinfowler.com/articles/continuousIntegration.html>

4. Basic writing and formatting syntax // GitHub Docs. – Режим доступу: <https://docs.github.com/en/get-started/writing-on-github/getting-started-with-writing-and-formatting-on-github/basic-writing-and-formatting-syntax>

Тривалість заняття: 2 год.